lsqvce

Least-squares adjustment, stochastic error modelling and variance component estimation

This is the single manual for the toolbox. It replaces the former user manual, reference manual and interpretation guide, which are consolidated here.

The toolbox solves large least-squares problems in batches, estimates the noise model from the residuals rather than assuming it, and re-weights observation groups by variance component estimation. It is built for the case where the data do not fit in memory, the noise is coloured and unknown, and the relative weighting of the observation types is exactly what you are trying to find out.

Version	3.0
Built	2026-08-31
Test suite	green on every push (CI); counts in README, audit check 17
User surface	lsqvce.fit, lsqvce.Options, lsqvce.Results, lsqvce.report
Engines	lsqvce.engine.adjust / adjustBlock / adjustVCE / adjustVCEjoint
Start your own case	examples/demo_fit.m and DATA_CONTRACT.md

Generated with assistance from Claude (Anthropic), model Claude Opus 4.8. Equations are drawn directly; workflow diagrams are generated from a single routine so every entry point is drawn the same way.

Contents

0 Start here: the 3.0 user surface

0.1 Three lines

0.2 The three kinds of data

0.3 buildDay in six rules

0.4 Reading the answer

D The data contract (v3) -- the binding specification

D.1 The principle: state nothing the data already states

D.2 The data struct, field by field

D.3 The buildDay contract

D.4 Arc schedules

D.5 Covariance specs

D.6 Local (arc-wise) unknowns

D.7 Multi-axis streams and the epochCovariance hook

D.8 Worked example: two satellites, ragged arcs

D.9 The equivalent engine calls

D.10 Checklist, and the two in-memory kinds

1 Orientation: which engine, and why (ENGINE tier)

1.1 The three engines fit dispatches to

1.2 Choosing between them

2 Theory

2.0 Where the theory lives: lsqvce_theory.pdf (R4-D3)

3 The entry points, step by step

3.1 adjust -- flat, in memory

3.2 adjustBlock -- block-arrow, one solve

3.3 adjustVCE -- the iterative loop

3.4 snoop -- outlier rejection

3.5 Reading a Results object

4 Adapting the template: a step-by-step guide (ENGINE tier)

4.1 What you write, and what the toolbox derives

4.2 Step 1 -- declare the processing options

4.3 Step 2 -- describe the observation groups

4.4 Step 3 -- write buildDay

4.5 Step 4 -- the a-priori covariance

4.6 Step 5 -- epochCov, for multi-axis streams

4.7 Step 6 -- run it, and read the result

4.8 Step 7 -- optional plot hooks

4.9 Four ways to get it wrong

5 Interpreting the result

5.1 The order to read things

5.2 Two ways least squares lies to you

5.3 The quadrant you must inspect

5.4 Deciding what to do next

5.5 Reading a gravity solution

6 Reference

6.1 Installation

6.2 Symbols

6.3 Equation summary

6.4 Error identifiers

6.5 Testing and the suite

R References

0 Start here: the 3.0 user surface

Since 2.0 the toolbox has four user-facing names: `lsqvce.fit` (the one verb), `lsqvce.Options` (what to do), `lsqvce.Results` (what came back) and `lsqvce.report` (the verdict). You describe the problem in a plain struct, **data**. Everything that does the work lives one level down, in `lsqvce.engine.*` and `lsqvce.kernel.*`, and is what the rest of this manual describes for experts.

0.1 Three lines

```
opts = lsqvce.Options(Model="vce");      % what to do: "gls" one solve, "vce" the loop
R    = lsqvce.fit(data, opts);           % the one verb: your data struct -> a Results
R.report                                 % the five-level verdict
```

0.2 The three kinds of data

`fit` tells them apart by the fields present; every rule is checked before any data is read, and an unknown field name is an error on purpose.

You have	The data struct	Engine fit runs
A design matrix in memory, diagonal weights, no arc-wise unknowns	data.A (m x u), data.l (m x 1, observed minus computed), optional data.p (weights)	adjust; Robust="irls" -> robust
The same, batched or nonlinear	data.batchFcn, data.numBatches, data.numUnknowns (linear form); with data.x0 the callback returns the Jacobian at x and u = numel(x0)	adjust / robust / solveNonlinear
Days, arcs, arc-wise unknowns, coloured noise (GRACE-FO)	data.commonParams (one row per common unknown), data.days, data.systems(k) with your buildDay and covariance (its fieldnames are the observation types); arcs / localsPerArc / localParams as needed	adjustBlock (gls), adjustVCE (vce, pooled), adjustVCEjoint (ellmer)

```
data.commonParams = commonTbl;            % one ROW per common unknown: nc = rows
data.days = 1:30;
s = struct();
s.buildDay         = @buildDayC;          % the one function you write
s.covariance.kbr   = struct('type','arma','ar',[1 -1.6 0.9 -0.25],'ma',1,'sigma',1e-7);
s.covariance.pos   = struct('type','diag','sigma',0.02);   % fieldnames = the types
s.id               = 'C';
s.arcs             = @(day, obsType) arcSchedule('C', day);   % ragged: ids per day
s.localsPerArc     = 6;                   % fit numbers the local blocks for you
data.systems(1) = s;                      % add satellite D as data.systems(2)
```

0.3 buildDay in six rules

`buildDay(day, obsType, ...)` returns, for one day and one observation type, a struct with A (m x nc), B (m x nl, may be sparse), l (observed minus computed), t (seconds) and, when the day carries several arcs, arcIdx / arcId.

1. **Deterministic.** Same arguments, identical output -- the data is read twice per pass.
2. **Raw, not whitened.** The toolbox whitens per arc from your covariance spec; the VCE loop reweights there.
3. **l is O minus C**, and t is in seconds.
4. **Arcs by id when ragged.** A resolver in arcs plus arcId in the day; a missing id is a data gap.
5. **Local blocks from arcInfo.** Fill B(rows, off + (1:k)); never compute an offset yourself.
6. **A stream called pos is 3-D by default.** A scalar stream with that name needs ComponentsPerEpoch = struct('pos', 1).

0.4 Reading the answer

```
R.report                % level-0..4 verdict with actionable findings
R.coefficients()        % estimates, formal errors, 95 % bounds as a table
R.x, R.sigma0           % the estimate and the a-posteriori sigma0
R.history, R.noise      % the VCE trajectory and the converged noise model
R.meta                  % provenance: version, release, UTC time, seeds, the options used
```

> A sigma0 of NaN is a result, not a bug: the quadratic form cancelled catastrophically and the toolbox refuses to invent a plausible number (lsqvce_theory.pdf, section 7).

Where to go next: chapter D (the data contract in full; DATA_CONTRACT.md in the repository is its source), examples/demo_fit.m (the three lines on every data kind, copy it), doc lsqvce.Options (every knob), lsqvce_theory.pdf (the method).

D The data contract (v3) -- the binding specification

This chapter is the manual's copy of DATA_CONTRACT.md (version 3, adopted 2026-08-31, Level L10): what you give `lsqvce.fit`, and how to write `buildDay`. The repository file is the binding source; the two are kept in step by the release process.

D.1 The principle: state nothing the data already states

The contract declares **structure**, never **counts**. Every count is derived from the one place it already lives:

Quantity	Derived from
nc (common unknowns)	size(data.commonParams, 1)
nl (local unknowns)	size(localParams, 1), or the localsPerArc plan; neither declared -> nl = 0
observation types	fieldnames(covariance), in declaration order
u (batched nonlinear)	numel(x0)

The v2 count fields (numCommon, numLocal, numUnknowns-with-x0) and the duplicate type list (observationTypes) are gone. One source each -- nothing left to disagree. A v2 declaration fails loudly: the retired names are unknown fields now, and `lsqvce:fit:unknownField` names the field.

D.2 The data struct, field by field

`data` is a scalar struct; field names are case-sensitive lowerCamelCase, unknown names are errors on purpose.

Field	Type / size	Meaning
commonParams	cell, R x C	REQUIRED. One row per COMMON unknown: nc = R. Column count and meaning are your convention; buildDay receives the table as its 3rd argument. Minimal: num2cell((1:nc).')
days	double, 1 x D	default day list (numeric integers; the cache is keyed on them); required unless every system carries its own
systems	struct, 1 x M	one entry per observing system (below)

Each `data.systems(k)` -- two required fields, the rest optional (fill unused optionals with []):

Field	Default	Meaning
buildDay	REQUIRED	your day builder (D.3)
covariance	REQUIRED	struct, one field per observation type holding a covariance spec (D.5). The fieldnames ARE the observation types; their order is the processing order
arcs	1	which arcs each day carries (D.4); 'fromDay' is ENGINE-tier only
localsPerArc	[]	the toolbox numbers the per-arc unknowns (D.6); XOR localParams
localParams	{}	R x C cell, one row per LOCAL unknown (nl = R), buildDay's 4th argument. Neither this nor localsPerArc -> nl = 0
days	data.days	per-system schedule override
id	the index k	numeric scalar or single char, unique across systems (explicit ids and index defaults share one namespace)
name	''	human-readable label for reports
processingOptions	struct()	anything buildDay needs; its 5th argument
epochCovariance	[]	@(day,obsType) per-epoch D hook for multi-axis streams (D.7)

> struct() with cell values expands into a struct ARRAY: s = struct('localParams', {'bias','drift'}) is 1 x 2. Use double braces, or assign field by field.

D.3 The buildDay contract

fit calls buildDay once per (day, observationType) and cuts the day into arcs itself. Every pass reads each day twice (normals, then residuals); the VCE loop multiplies that by the passes. Hence: **deterministic** (same arguments, byte-identical output -- no randomness, clocks, mutable globals), and cache expensive raw reads yourself (the toolbox caches one built day).

fit passes as many of these six arguments as the handle declares (nargin); a varargin builder receives all six:

```
D = buildDay(day, obsType)                                  % classic
D = buildDay(day, obsType, common, local, procOpt)          % parameter-driven
D = buildDay(day, obsType, common, local, procOpt, arcInfo) % with local-block plan
```

Return raw, unwhitened matrices -- whitening happens per arc inside the toolbox, and the VCE loop reweights there. Required fields: A (m x nc, common design), B (m x nl, local design, may be sparse), l (m x 1, OBSERVED MINUS COMPUTED), t (m x 1, seconds). Optional: arcIdx (cell of row-index vectors; absent = one arc), arcId (the identity of each arc for ragged schedules), nAxis and Cep for multi-axis streams (D.7).

Declaration errors are raised by fit BEFORE any data is read (lsqvce:fit:unknownField / noSystems / noCommon / missingField / localCount / noDays / badId / duplicateSystem / builderKind / badArcs, plus every makeLoader construction id relabelled under lsqvce:fit with the original as the cause). Reading errors keep their engine ids: lsqvce:makeLoader:buildDay / badDay / dayShape / badArc / badArcId, lsqvce:accum:arcBadNc / arcBadNl -- the derived nc and nl are enforced against every day.

D.4 Arc schedules

Static -- every day looks the same: arcs = 4 (ids 1..4), arcs = [1 2 4], or per type arcs.kbr = 2. Arcs are addressed by position in arcIdx; an id beyond numel(arcIdx) is an error, which catches a mis-declared schedule.

Ragged -- days differ (gaps, manoeuvres, short months): declare a resolver that returns the arc ids of a day without reading data, and let buildDay return arcId naming the arcs it actually carries. Arcs are then addressed by id; an id a day does not carry is a data gap and is skipped -- no padding, no empty arcs.

```
s.arcs = @(day, obsType) myArcTable{day};   % 1 x A vector of ids for that day
```

D.5 Covariance specs

One struct per observation type; type selects the model. sigma is always a STANDARD DEVIATION, never a variance; the VCE loop scales the specs at run time, you give the starting point. Full reference: doc lsqvce.engine.covModel.

type	Fields	Use for
'diag'	sigma (scalar or m x 1 std)	uncorrelated noise
'arma'	ar [1 a1 ... ap], ma (default 1), sigma	coloured noise with a known filter
'acf'	acf @(dt,opt), sigma	correlation as a function of the lag
'psd'	f (nf x 1, Hz), S (nf x 1)	noise given as a one-sided PSD
'var'	ar (d x d x p), sigma (d x d)	vector-AR for multi-axis streams

D.6 Local (arc-wise) unknowns

nl has two declared sources and one default -- never more than one applies. **localsPerArc** (a count or @(day,arcId)): the toolbox numbers every (day, arc) over the whole adjustment, derives nl, and hands each day its slice as arcInfo -- fill B(rows, off + (1:k)) from arcInfo.offsetOf(arcId) and never compute an offset yourself. **localParams** (R x C cell): you own the numbering, nl = R, the table is buildDay's 4th argument. Declaring both is lsqvce:fit:localCount. **Neither**: nl = 0 -- return B = zeros(m, 0); the engine skips the pre-elimination and the local diagnostics come back as consistent empty shapes.

D.7 Multi-axis streams and the epochCovariance hook

A kinematic position has d = 3 components per epoch sharing one temporal law. Return the rows EPOCH-MAJOR (x1 y1 z1 x2 y2 z2 ...), set nAxis = 3, and provide the per-epoch scaling D. The STANDARD source is Cep on the returned day

(nEpoch x d, or nEpoch x d x d for full per-epoch blocks); a static scaling can ride on the covariance spec instead. A multi-axis stream with no D is an error.

The **epochCovariance hook** is optional and matters only under Model="vce": whitening never needs it, but the Shape step of the VCE iteration (re-estimating the temporal AR law from residuals) does not read the day's Cep. With the hook it takes the exact separable de-scale path (lsqvce.kernel.residualAcv with the full per-epoch D, fetched per day WITHOUT rebuilding the day); without it, it falls back to descaling by the stream's current scalar sigma -- an approximation that biases the estimated AR law when D varies strongly across epochs (changing geometry / PDOP). Declare the hook when you run VCE on multi-axis streams and have D available independently of the day build.

D.8 Worked example: two satellites, ragged arcs, per-arc locals

```
function D = buildDayGrace(day, obsType, common, ~, procOpt, arcInfo)
    raw = readGraceDay(procOpt.dataDir, day, obsType);   % your reader, deterministic
    D.A = shPartials(raw, common);                       % m x nc
    D.l = raw.observed - raw.computed;                   % O - C
    D.t = raw.tSeconds;
    D.arcIdx = raw.arcRows;   D.arcId = raw.arcIds;
    D.B = sparse(numel(D.l), arcInfo.nl);
    for a = 1:numel(D.arcId)
        off = arcInfo.offsetOf(D.arcId(a));
        D.B(D.arcIdx{a}, off + (1:arcInfo.count(a))) = localPartials(raw, a);
    end
end
```

```
data.commonParams = shParameterTable(60);      % nc = rows, columns yours
data.days = 1:31;
cov.kbr = struct('type','arma','ar',[1 -1.6 0.9 -0.25],'ma',1,'sigma',1e-7);
cov.pos = struct('type','diag','sigma',0.02);  % fieldnames = the two types
s = struct();
s.buildDay = @buildDayGrace;   s.covariance = cov;
s.id = 'C';   s.name = 'GRACE-FO C';
s.arcs = @(day, obsType) arcSchedule('C', day);
s.localsPerArc = 6;
s.processingOptions = struct('dataDir', '/data/gracefo');
data.systems(1) = s;
s.id = 'D';  s.name = 'GRACE-FO D';  s.arcs = @(day, obsType) arcSchedule('D', day);
data.systems(2) = s;
R = lsqvce.fit(data, lsqvce.Options(Model="vce", GroupBy="satObs", Parallel="parfor"));
R.report
```

D.9 The equivalent engine calls

fit builds one Loader per system through lsqvce.engine.makeLoader (field names map 1:1 to its Name-Value names, lowerCamelCase to UpperCamelCase, plus the D.1 derivations) and dispatches on opts.Model to adjustBlock / adjustVCE / adjustVCEjoint. These are ENGINE-tier calls; a user needs none of them. The ENGINE tier keeps the v2 conveniences (numCommon/numLocal counts, 'fromDay', cache, plot hooks, raw loader closures) -- they left the USER surface, not the toolbox.

D.10 Checklist, and the two in-memory kinds

1. buildDay returns A, B, l, t with one row count; l is O - C; nothing is whitened.
2. Calling buildDay twice with the same arguments gives identical output; t is in seconds.
3. commonParams has one row per common unknown -- size(A, 2) must equal it.
4. Every observation type is a field of covariance; the fieldnames are the type list.
5. Ragged schedule -> resolver + arcId; static -> count/vector, no arcId.
6. Locals: localsPerArc (fill B from arcInfo) or localParams (you number); neither -> B = zeros(m, 0).
7. Multi-axis: epoch-major rows, nAxis, Cep on the day; the epochCovariance hook for the exact VCE Shape path.
8. No unknown field names -- a typo is an error, and so is every v2 count field.

Not every problem streams. When A fits in memory and there are no arc-wise unknowns, data has no systems field at all; fit tells the kinds apart by the fields present. **Kind B**: A (m x u), l (m x 1, O - C), optional p (weights). **Kind C**: batchFcn with [A, l, p] = batchFcn(k), numBatches, and numUnknowns on the LINEAR form; with x0 the problem is nonlinear ([A, w, p] = batchFcn(k, x), Gauss-Newton) and u = numel(x0) -- numUnknowns must then be absent (lsqvce:fit:uCount). Model must be "gls" on both kinds; Robust="irls" selects M-estimation. The same Results comes back; residuals() and contribution() refuse without an accumulator and say so.

> A stream whose observation type is called pos is treated as 3 components per epoch by default, which selects the separable model and needs a per-epoch scaling. A scalar stream that happens to be called pos must declare ComponentsPerEpoch = struct('pos', 1).

1 Orientation: which engine, and why (ENGINE tier)

Three functions solve; everything else diagnoses, models or accumulates. Picking the right one is the first decision, and it follows from two properties of your problem: whether the normal matrix has block-arrow structure, and whether you already know the noise model.

> Since 2.0 (R4, 2026-09-01) the front door is ONE verb: **lsqvce.fit(data, opts)** -- see the Getting started page (doc lsqvce) and DATA_CONTRACT.md. The three engines below are what fit dispatches to (lsqvce.engine.adjust / adjustBlock / adjustVCE); this part describes them for experts and for readers of 1.x code. Every name in this manual that is not fit, Options, Results or report is now qualified with lsqvce.engine. or lsqvce.kernel. (RELEASE_NOTES.md).

1.1 The three engines fit dispatches to

Function	Structure	Noise model	Data
adjust	none assumed; one dense normal matrix	**required** -- you supply the covariance	all in memory at once
adjustBlock	block-arrow: many local blocks, one common block	**required** -- you supply it	accumulated in batches
adjustVCE	block-arrow, as above	**estimated from the data**	accumulated, several passes

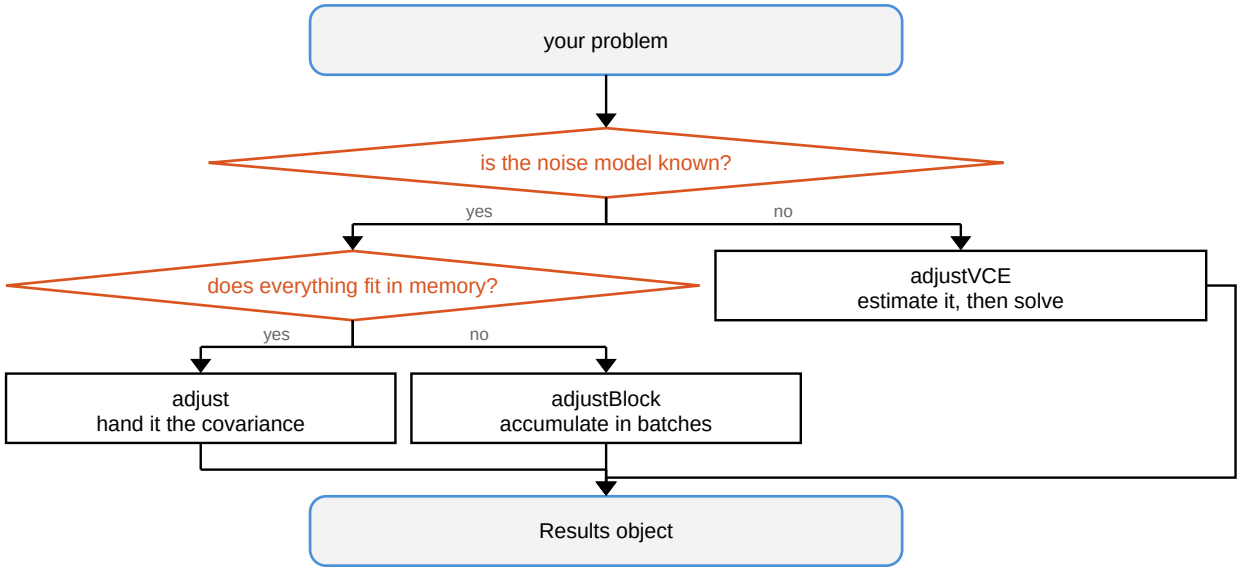
Table 1.1 -- The three solving entry points. adjustVCE is a fluent builder over adjustBlock: it runs the same solve repeatedly, re-estimating the noise model and the group weights between passes.

1.2 Choosing between them

Two questions decide it, in order. First: **is the noise model known?** If it is not, it has to be estimated from the residuals, which is iterative, and **adjustVCE** is the only engine that does it. Nothing else about the problem matters yet.

If the model *is* known, the second question is size: does everything -- design matrix, observations and the full covariance -- fit in memory at once? If it does, **adjust** takes it directly. If it does not, the normal equations must be accumulated batch by batch, which is **adjustBlock**.

Figure 1.1 -- choosing an engine



Diamonds are decisions, and the branches are labelled. A known noise model is a PREREQUISITE for both adjust and adjustBlock: each takes the covariance as given. What separates those two is only size. adjustVCE is the answer whenever the model is not known, regardless of size, because estimating it is inherently iterative.

> **If you are unsure, start with adjustBlock and a plain white a-priori covariance.** Run it, read the report, and look at whether the residuals are white. If they are not, that is exactly the condition adjustVCE exists to fix, and you can switch by adding two fluent calls to the same loaders.

2 Theory

The theory of the toolbox -- the Gauss-Markov model and its two backends, conditioning and regularisation, variance component estimation (Helmert, MINQUE, LS-VCE, Monte-Carlo), the separable stochastic model and the three iteration loops, robust estimation, the spherical-harmonic setting, error handling, the workflow and the method's limitations -- is documented ONCE, in **lsqvce_theory.pdf** at the toolbox root (rebuild: *python3 dev/docs_src/build_theory_pdf.py*). Every claim there names the work package that pinned it in HANDOVER.md.

> Decision R4-D3 (2026-09-01): this manual no longer carries a theory chapter of its own; the former sections 2.1-2.7 were retired so that no statement exists in two places. Where the practitioner chapters cite an equation, the number refers to the theory document (1.1-7.1).

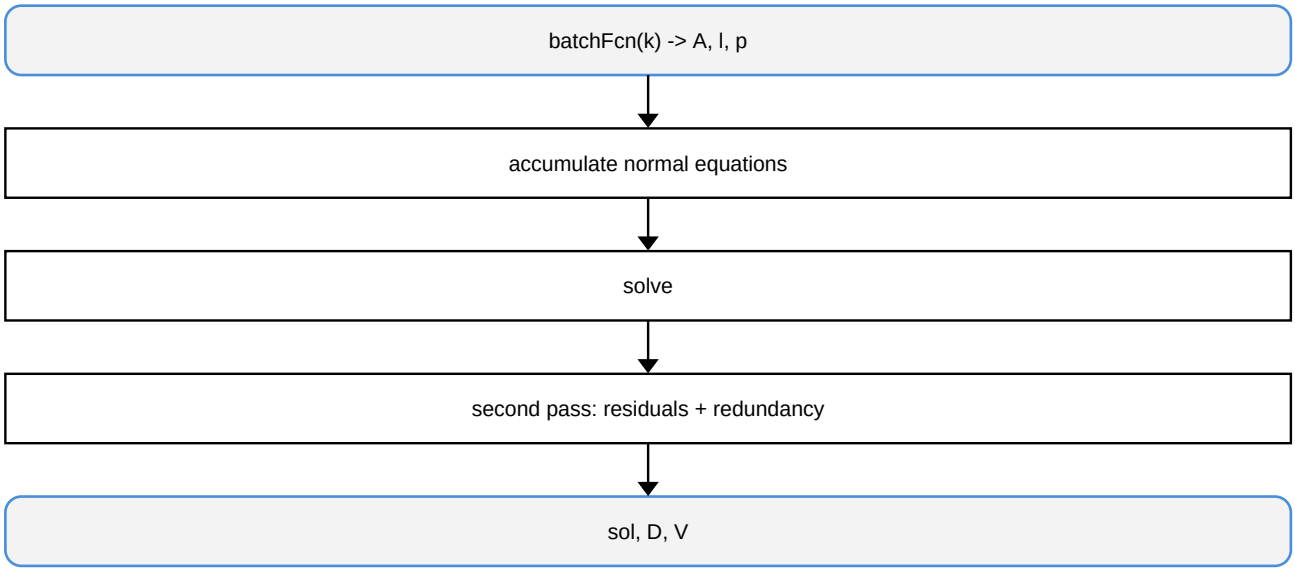
3 The entry points, step by step

Each engine is shown as a workflow: what you supply, what the toolbox does with it, and what comes back. The shaded rounded boxes are yours; the plain boxes are the toolbox.

3.1 adjust -- flat, in memory

The direct path. You supply a batch function that hands over design rows, observations and weights; **adjust** accumulates the normal equations, solves, and returns the solution together with its diagnostics. One pass, no iteration, no assumed structure.

Figure 3.1 -- adjust



batchFcn is called TWICE -- once to accumulate, once to diagnose -- and must return identical data both times. A mismatch is an error, not a silent misattribution.

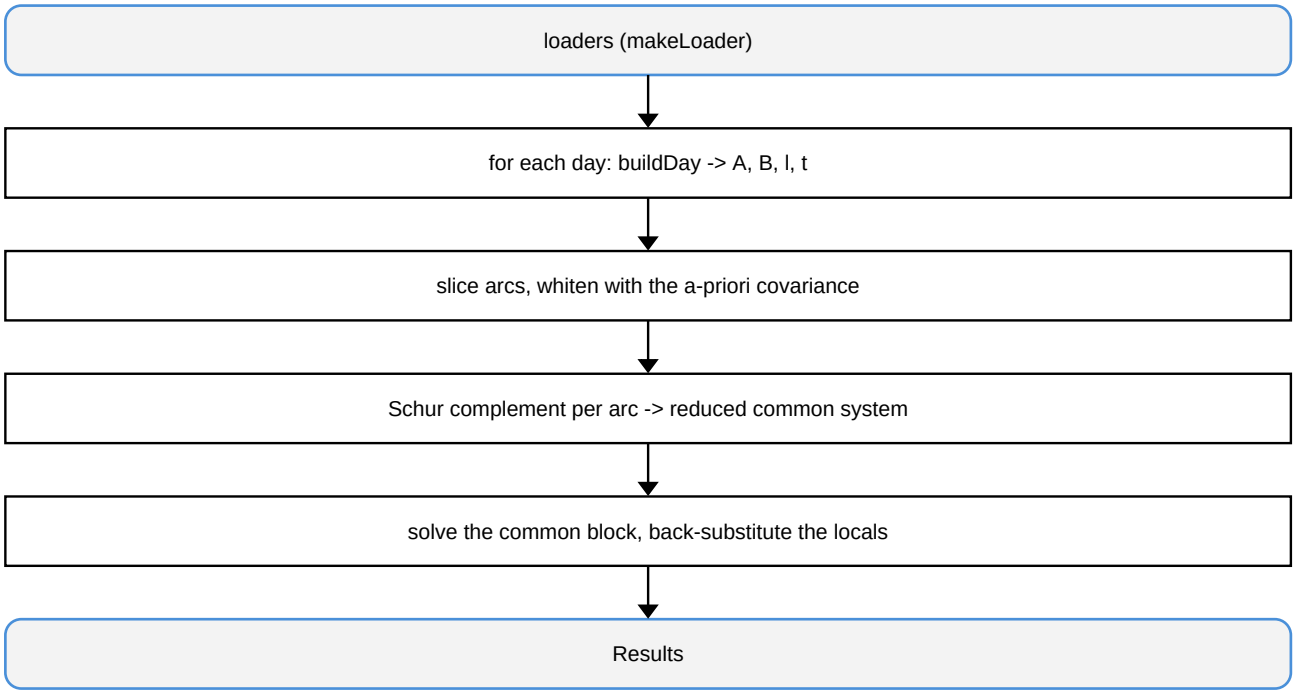
```
[sol, D, V] = lsqvce.engine.adjust(u, batchFcn, nBatch);
lsqvce.report(V);                % the level-0..4 verdict
```

The returns are structs, not a Results object: `sol` carries the estimate and its dispersion, `D` the diagnostics, `V` the verdict. Because there is no iteration there is no history, so the iteration readers of section 3.5 do not apply here.

3.2 adjustBlock -- block-arrow, one solve

The same single solve, but exploiting block-arrow structure and streaming the data. You supply *loaders* rather than a batch function; each loader knows how to build one day of one observation type. The accumulator slices arcs, applies the a-priori covariance, forms each arc's Schur complement and adds it to the reduced common system.

Figure 3.2 -- adjustBlock




```
R = lsqvce.engine.adjustBlock({Lc, Ld}, days);
R.report();
```

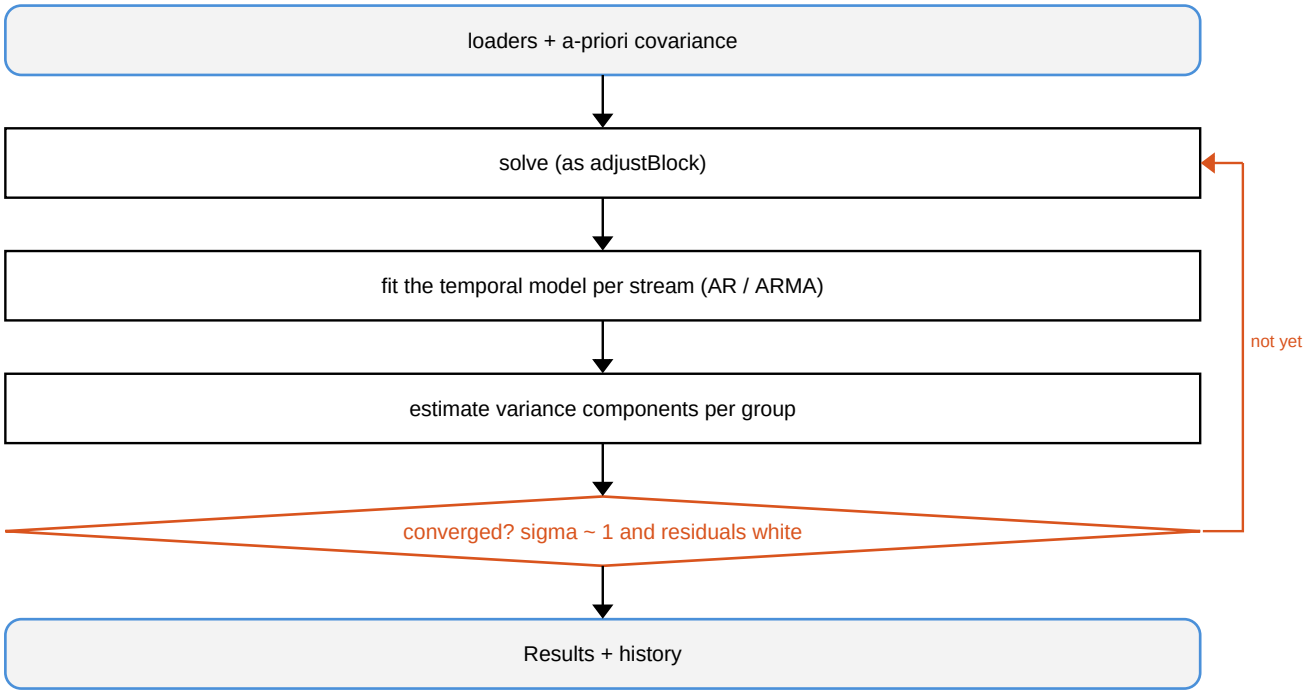
> **The noise model is taken as given.** Whatever covariance you declared on the loaders is used as-is. If the residuals come out coloured, adjustBlock will not tell you to fix it -- that is what adjustVCE is for.

Option names and typed signatures (v2.1, WP-39). Public entry points are migrating from inputParser to typed arguments blocks. Three things change for callers: (i) every Name-Value option has ONE canonical UpperCamelCase name, stated in the function help (Backend, MaxIter, Verbose, …); (ii) name matching itself remains case-insensitive (MATLAB name-value semantics, verified on CI), so 'backend' or 'maxiter' keep working -- write the canonical casing anyway; (iii) enum VALUES are case-sensitive lowercase ('qr', not 'QR') and unknown option names are rejected instead of ignored. Migrated so far: adjustBlock, adjustVCEjoint, iterateNoiseModel; the remaining surface follows in the same release (see the release notes in the repository history).

3.3 adjustVCE -- the iterative loop

The same solve as adjustBlock, run repeatedly, with two estimation steps between passes. After each solve the residuals are examined: a temporal noise model is fitted per stream (the *shape* of the noise), and the variance components are re-estimated per group (its *scale*). Both feed back into the next solve. The loop stops when the variance factors are near one and the residuals test white.

Figure 3.3 -- adjustVCE



The orange arrow is the outer loop. Each pass re-solves with the noise model and weights discovered by the previous one; R.history records every pass, which is what the two readers in section 3.5 report on.

```
R = lsqvce.engine.adjustVCE(L, days, 'GroupBy', 'obsType').run();
R.report();
R.summary();          % per stream: what it ended up as
R.convergence();      % per iteration: when it settled
```

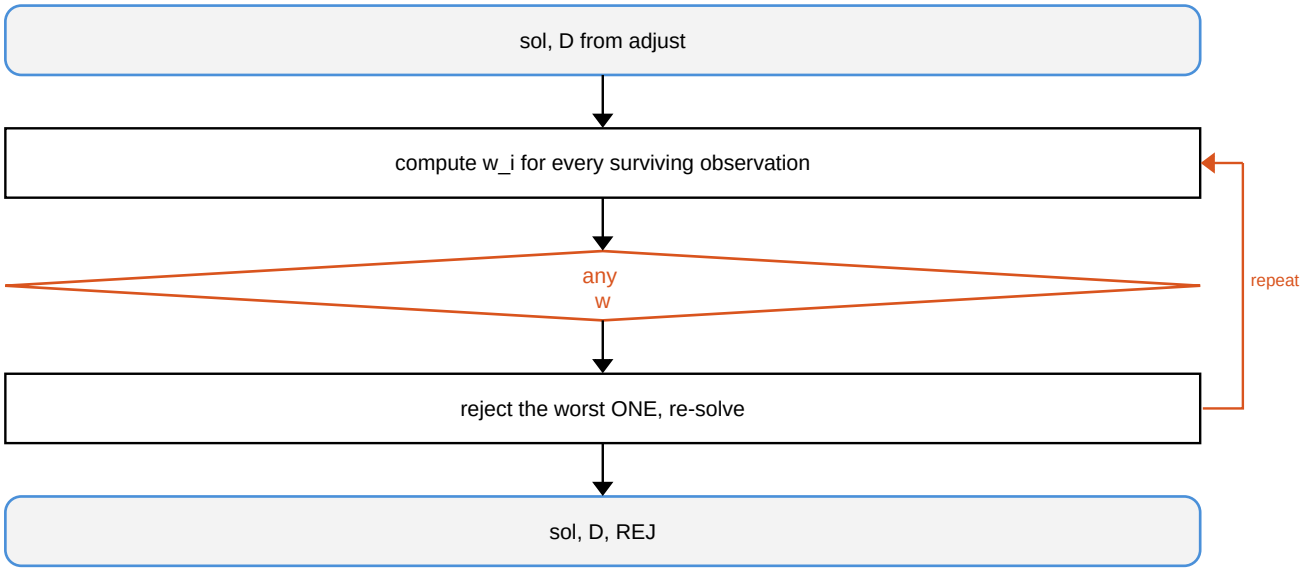
The constructor groups configure the loop before it runs (constructor-only since 2.0, R4-D2). `GroupBy` sets the granularity of the variance components; the `Shape` and `Scale` structs tune the two estimation steps. All have documented defaults -- start without them and change one only when a diagnostic tells you to. Through the USER front door the same groups come from `lsqvce.Options`.

3.4 snoop -- outlier rejection

> **snoop is a SEPARATE step, and it works on the flat path only.** It is not part of any chain in 3.1-3.3: no engine calls it, and `adjustBlock` and `adjustVCE` have no equivalent. Its signature says so -- it takes `sol` and `D`, the structs `adjust` returns, plus the same `batchFcn`, because it must re-solve after every rejection.

Data snooping tests each residual against its own standard deviation (equation 2.7), rejects the single worst, re-solves, and repeats until nothing trips the critical value.

Figure 3.4 -- snoop, an iteration around adjust



One at a time, and the reason is MASKING: several blunders inflate s0 together, so each hides the others -- every |w| is divided by a variance factor the blunders themselves raised. Remove one, s0 drops, and the next becomes visible.

```
[sol, D]      = lsqvce.engine.adjust(u, batchFcn, nBatch);     % first solve
[sol, D, REJ] = lsqvce.engine.snoop(sol, D, batchFcn, 'test', 'mad');
```

On the block-arrow path there is no snoop. Gross-error handling there is a different problem: the observations are streamed and never all resident, and an arc whose local block is near-singular is a model defect rather than an outlier. Read the verdict from `R.report()` and the per-stream state from `R.summary()`, and treat a stream that will not whiten as a modelling question -- section 4.9.

Localise before rejecting. A structured error is a model defect, and snooping it away mutilates good data. Run `lsqvce.engine.systematic(D)` first: if the residuals show a pattern, fix the model, not the observations. And read section 5.3 before trusting any rejection at low redundancy.

3.5 Reading a Results object

adjustBlock and **adjustVCE** both return a `lsqvce.Results`. It carries the solution, the accumulator, and -- for the iterative engine -- the full history.

Call	Returns
`R.report()`	the level-0..4 verdict; read it top-down and stop at the first red
`R.redundancy()`	total redundancy, $n - u$
`R.residuals(...)`	whitened or raw residuals
`R.contribution(...)`	per-group information share of the solution
`R.summary()`	one row per **stream**: final sigma, AR order, whiteness
`R.convergence()`	one row per **iteration**: when the loop settled

The two iteration readers are complementary, not duplicates. `convergence()` reduces across streams (one row per pass); `summary()` reduces across passes (one row per stream). A loop that converges can still hide one stream that never whitened -- the per-pass view reports the worst p-value without naming its owner. Both need a history, so both raise `lsqvce:Results:noHistory` on a single solve.

4 Adapting the template: a step-by-step guide (ENGINE tier)

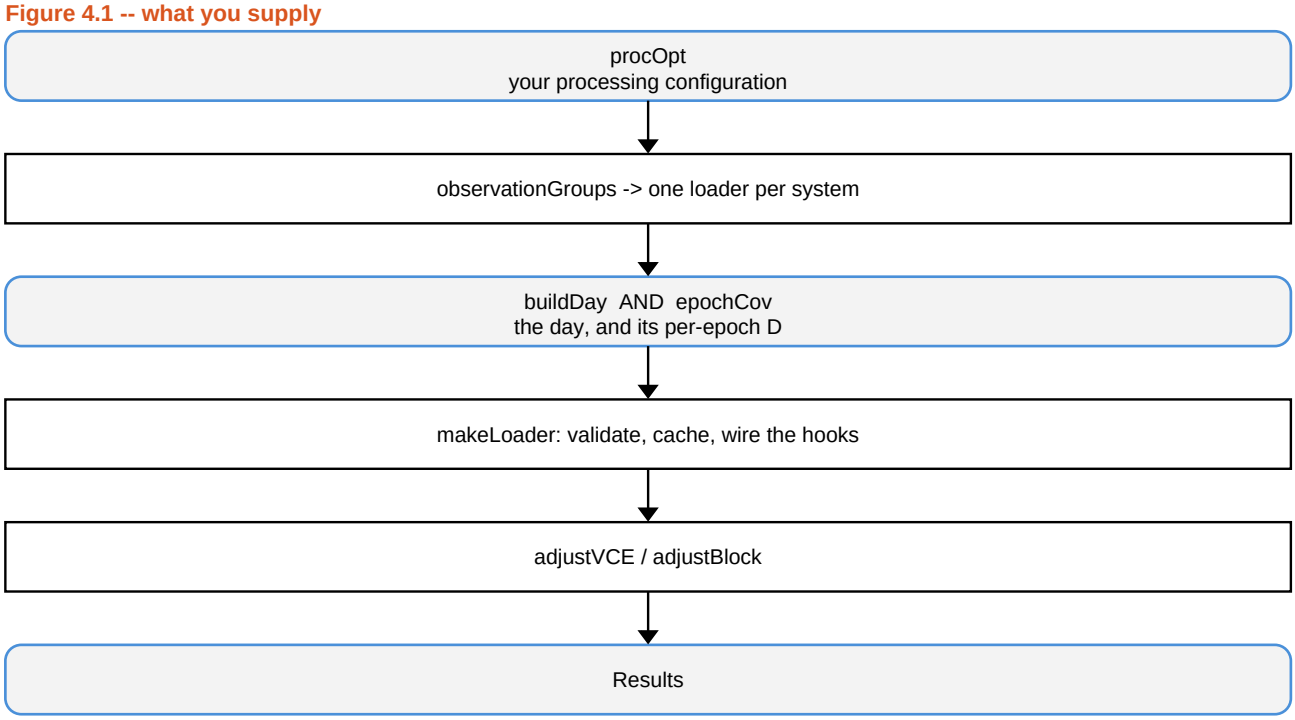
> Since 2.0 the user path needs no template and no Loader: describe the case as a data struct (chapter 0, DATA_CONTRACT.md) and call lsqvce.fit. Steps 3 (buildDay), 4 (the a-priori covariance) and 5 (epochCov) below are the same work in both worlds -- the data struct carries them as buildDay, covariance and epochCovariance. Keep the template when you want the Loaders in hand (procOpt-driven cases, direct engine work, 1.x code).

Copy `template/lsqvce_template.m`, rename it, and work through the seven steps below. The template is written so that the parts you must replace fail loudly until you do: the two data stubs raise `lsqvce_template:notImplemented` rather than returning plausible fake data.

> **The template has two registers.** The loader loop is real, runnable code -- it builds one loader per observation group and you edit it only if your topology differs. `myBuildDay` and `myEpochCov` are stubs that error until written. Running the file as shipped therefore gets you loaders that fail on first use; that is the expected state, not a defect.

4.1 What you write, and what the toolbox derives

Figure 4.1 -- what you supply



Rounded boxes are yours; everything below makeLoader is the toolbox's job. buildDay and epochCov are supplied TOGETHER, not as alternatives: buildDay gives the design and the observations, epochCov gives the per-epoch spatial covariance D for the same rows. A scalar stream is the one case where epochCov is omitted -- there is no D to supply when d = 1.

You state	Where	The toolbox derives
the common parametrisation	`commonParams` (one row per unknown)	`nc`, from the row count
the local parametrisation	`localParams`, per system	`nl`, from the row count
which streams exist	`observationTypes`	the variance-component groups
the day list	`procOpt.days`	the batches to stream

> **State each value once.** There is no `nc` argument on any entry point: the common-parameter count is stated where the parametrisation is defined and derived everywhere else, so the two cannot drift apart. The same discipline applies to your own configuration -- feed the toolbox *from* it rather than restating values.

4.2 Step 1 -- declare the processing options

`procOpt` is your configuration. The toolbox never reads a field of it: it is captured when the loader is built, piped unchanged to your `buildDay` as its fifth argument, and surfaced to plot hooks as `ctx.procOpt`. Group it into nested structs; only the fields your builder needs have to exist.

```
procOpt.days          = 1:31;
procOpt.loggingLevel  = 1;              % 0 silent | 1 progress | 2 detail
procOpt.data          = struct('pathToLoad', ..., 'maximumDegree', 60);
procOpt.epochs        = struct('mEpoch', 300, 'dt', 5, 'arcsPerDay', 2);
procOpt.forces        = ...;            % your own sub-structs, opaque to lsqvce
procOpt.observationGroups = { groupC, groupD, groupE };
```

> **procOpt is immutable input, not shared state.** It is frozen when `makeLoader` is called. A live cursor such as an iteration index would stay at its initial value while the loop iterates around it, and your builder would silently read the wrong pass forever. The engine tracks these itself: use `ctx.pass` and `ctx.sysID`.

Values your pipeline needs before any toolbox object exists stay in `procOpt`, and the toolbox is fed *from* them:

```
R = lsqvce.engine.adjustVCE(L, procOpt.days, ...
        'verbose', procOpt.loggingLevel, 'GroupBy', 'obsType').run();
```

Note that `loggingLevel` maps directly onto `verbose`, which accepts 0, 1 or 2. **Level 3 is rejected**, so a pipeline that can emit 3 must clamp before passing it.

4.3 Step 2 -- describe the observation groups

Each entry of `observationGroups` describes one *system*: which streams it carries and how it is identified. The template loops over them and calls `makeLoader` once per group. This is where the topology is fixed.

> **When two satellites must share one system.** Merge them whenever an observation's design couples both satellites' *local* parameters -- an inter-satellite range or range-rate does exactly this. The block-arrow model has one local block per system (equation 2.10), so a coupling observation cannot span two of them: give both satellites' streams the same group and stack their locals. Streams that are merely *correlated* do not need merging; that is what the variance components are for.

> `systemId` **must be numeric or a single character.** The accumulator indexes systems by `[store.sysID]`, which flattens a multi-character string into separate elements. Carry a readable label in `'name'` instead.

4.4 Step 3 -- write buildDay

This is the one function nobody can write for you. It reads one day of one observation type and returns it in the shape the accumulator expects.

```
function D = myBuildDay(day, obsType, common, local, procOpt)
    nc = size(common, 1);          % derive, never hardcode
    nl = size(local, 1);
    ...
    D.A      = ...;   % m x nc   common design matrix, WHOLE day
    D.B      = ...;   % m x nl   local design matrix (may be sparse)
    D.l      = ...;   % m x 1    observations
    D.t      = ...;   % m x 1    epoch timestamps [s]
    D.arcIdx = {1:n1, n1+1:n2};    % row indices, one cell per arc
    D.nAxis  = 3;                  % axes per epoch; omit for scalar streams
end
```

Field	Size	Required?
`.A`	$m \times nc$	yes
`.B`	$m \times nl$	yes
`.l`	$m \times 1$	yes
`.t`	$m \times 1$	yes
`.arcIdx`	1 × nArc cell	optional -- omitted means the whole day is ONE arc

Field	Size	Required?
`.nAxis`	scalar	multi-axis streams only

> **It must be deterministic.** The accumulator reads every day a *second* time for the residual pass and must get identical data back. Seed any random number generator from (`day`, `obsType`), never from the clock -- and beware a file being rewritten while the run is in progress.

You return a whole day, not an arc. The accumulator slices arcs out of what you return, using `.arcIdx`. Omitting it makes the whole day one arc, which is right for a single continuous pass and wrong for a day containing several -- one local block would then be asked to describe several disjoint states.

4.5 Step 4 -- the a-priori covariance

One covariance spec per observation type. If you are going to run **adjustVCE**, a deliberately plain start is honest: the loop exists to discover the truth, and beginning with unit-variance white noise lets you watch it work.

```
cov.pos = struct('type','separable', 'ar',1, 'sigma',1, 'd',3);
cov.rr  = struct('type','diag',      'sigma',1);
```

Scalar streams use `'diag'`; multi-axis streams use `'separable'`, the model of equation 2.12, and must also supply the per-epoch **D** described next.

> `'separable'` **here is a covariance MODEL, not an engine name.** It is one of eight values `type` can take -- alongside `diag`, `acf`, `psd`, `arma`, `var`, `toeplitz` and `custom` -- and it names the structure of equation 2.12, $C = \sigma^2 D(R_t \otimes I_d)D$. The *class* once called `SeparableProblem` was renamed `adjustVCE` in 2026; the covariance type was not touched, because it describes mathematics rather than an entry point.

4.6 Step 5 -- epochCov, for multi-axis streams

Only multi-axis streams need this. It supplies the known spatial half **D** of the separable model, per epoch, so the temporal law can be recovered by exact de-scaling rather than inferred from whitened residuals.

```
function dd = myEpochCov(day, obsType, procOpt)
    dd.Cep = ...;                  % nEpochDay x d   OR  nEpochDay x d x d
    dd.arc(1).epochs = 1:300;      % which day-epochs this arc covers
    dd.arc(1).rows   = 1:900;      % and which ROWS  (R = E*d)
end
```

The two accepted layouts of `Cep` are given in table 2.1: `nEpoch × d` for per-axis standard deviations, or `nEpoch × d × d` for the full per-epoch covariance. Anything else raises `lsqvce:covModel:badCep`.

> The `.arc(i).epochs` and `.arc(i).rows` fields exist so the two indexings can be *checked against each other* rather than assumed to agree. Row k belongs to epoch k/d (equation 2.13); if your rows and your epochs disagree, this is where it surfaces.

4.7 Step 6 -- run it, and read the result

```
L = myCase(procOpt);                    % the template returns loaders
R = lsqvce.engine.adjustVCE(L, procOpt.days, 'verbose', procOpt.loggingLevel, ...
        'GroupBy', 'obsType').run();

R.report();          % the verdict -- read top-down, stop at the first red
R.summary();         % per stream: sigma, AR order, whiteness
R.convergence();     % per iteration: did it settle, and when
```

Read `R.report()` first and stop at the first red. A Level 0 red is a data or weighting bug, not a blunder -- fix it before reading anything else. Only once the verdict is clean are the variance factors worth interpreting.

4.8 Step 7 -- optional plot hooks

A hook is a handle *you* write, `@(ctx) -> figureHandle`. It only *builds* a figure; the engine owns showing, exporting and closing it. A hook that throws is logged and skipped, so a broken plot never breaks a solve.

Stage	Fires	ctx additionally carries
`postSolve`	after EACH solve, before the residual pass	the current estimate
`postModel`	ONCE, after the stochastic model is fitted	`.psd .psdFreq .acv .arOrder`

> **Hooks attach at construction.** Passing a bare handle to `makeLoader` broadcasts it to every observation type. Assigning `L.plotPostSolve` *after* the loader exists requires a `containers.Map` keyed by obsType -- the engine skips anything else **silently**.

4.9 Four ways to get it wrong

Two of these the toolbox catches for you. Two it cannot, and those are the ones worth memorising.

I -- a builder that is not deterministic

```
% WRONG -- fresh noise on every call
D.l = readObs(day) + 0.01*randn(m,1);

% RIGHT -- seeded from (day, obsType), so both passes agree
rng(day*1000 + double(obsType(1)));
D.l = readObs(day) + 0.01*randn(m,1);
```

How you notice: you do not, from the numbers alone. Residuals are attributed to the wrong observations while every statistic still looks plausible.

II -- axis-major row order

```
% WRONG -- all x, then all y, then all z
D.A = [Ax; Ay; Az];

% RIGHT -- epoch-major: [x1 y1 z1 | x2 y2 z2 | ...]
D.A = reshape([Ax'; Ay'; Az'], nc, [])';
```

> **This is the silent one.** The temporal model strides by `nAxis` and equation 2.13 pairs each row with an epoch. Stored axis-major, every row is paired with the wrong epoch: a correlation length of 60 epochs is read as 180. The fit succeeds, the AR order looks sensible, the variance factors converge -- and the stochastic model is wrong. **Check row order first whenever a multi-axis stream will not whiten.**

III -- variances where weights are expected

The weight **P** is the *inverse* of the covariance. Passing σ^2 instead of $1/\sigma^2$ silently inverts the weighting, so your best observations are down-weighted. It is dimensionally valid, so nothing can detect it for you.

IV -- mismatched row counts

```
D.A = zeros(m, nc);   D.l = zeros(m+1, 1);   % A and l disagree
```

How you notice: immediately, and by name. `lsqvce:makeLoader:dayShape` reports the day and obsType; its sibling `lsqvce:makeLoader:badDay` fires when a required field is missing. These two are why a shape error never reaches the solver.

A checklist for a stream that will not whiten

1. **Row order** -- epoch-major? (Failure II: silent, and the most common cause.)
2. **Arcs** -- is `.arcIdx` really splitting the day, or did you inherit the whole-day default?
3. **AR order** -- is the fit pinned at `MaxOrder`? Read `R.summary()`; a stream at the ceiling needs a higher one, or ARMA.
4. **Grouping** -- does each group have enough redundancy? Oscillating variance factors in `R.convergence()` mean it does not; coarsen `GroupBy`.

5. **Determinism** -- re-run and compare. Different numbers from the same input means failure I.

5 Interpreting the result

This part is about judgement rather than syntax: which number to look at first, what a red flag means, and the two ways least squares will quietly mislead you. The equations behind it are in Part 2.

5.1 The order to read things

`R.report()` is organised in five levels. **Read it top-down and stop at the first failure** -- a break at one level makes every level below it meaningless.

Level	Reads	If red	Why it comes first
0 Integrity	the trace identity	a data or weighting bug	if $\sum r_i \neq n-u$ the geometry is wrong and nothing else is trustworthy
1 Global	model consistency	inconsistent	s_0^2/σ_0^2 too large means blunders or optimistic weights
2 Localise	find the culprit	snoop	but check the contamination gauge first
3 Reliability	what you cannot check	you are blind here	low r_i means unverifiable, whatever the residual
4 Parameters	result fragility	results fragile	high correlation or conditioning means weakly separated unknowns

Table 5.1 -- the five levels of the verdict. A Level 0 red is not a blunder; it is a bug in what you supplied.

Why Level 0 is an identity, not a test

The redundancy numbers are the diagonal of a projector (equation 2.5), and the trace of a projector equals its rank. So their sum is not an estimate that might be close -- it is fixed by the geometry:

$$\sum_i r_i = \mathrm{tr}(\mathbf{RQ}) = n - u \tag{5.1}$$

Any departure beyond rounding means the weights, the design matrix or the rank are not what the toolbox was told. That is why it is read first: no later number can be salvaged from a system whose geometry is misreported.

What Level 1 actually tests

The global test compares the a-posteriori variance factor with the a-priori one. Under the model, the weighted sum of squares is chi-square distributed with r degrees of freedom:

$$\mathbf{v}^T\mathbf{P}\mathbf{v} \,/\, \sigma_0^2 \sim \chi^2(r), \quad s_0^2 \,/\, \sigma_0^2 = \mathbf{v}^T\mathbf{P}\mathbf{v} \,/\, (r\,\sigma_0^2) \tag{5.2}$$

A ratio far above one has exactly three causes, and they are not distinguishable from this number alone: gross errors in the observations, weights that were too optimistic, or a model that is missing a term. That ambiguity is why Level 1 tells you to look further rather than what to do -- section 5.4 separates the three.

> **A ratio far BELOW one is also a failure**, and it is the one people ignore. It means the weights were pessimistic: the solution fits better than the stated precision allows. The estimate is not wrong, but every covariance derived from it is too large, so formal errors and MDBs are overstated and the network looks weaker than it is.

5.2 Two ways least squares lies to you

Lie 1 -- "the residual is small, so the observation is good"

False. A residual shows only the fraction r_i of the error present:

$$\mathrm{E}\{\Delta v_i\} = -r_i\, b_i \tag{5.3}$$

At $r_i \approx 0$ even a huge blunder leaves a near-zero residual. **Always pair a residual with its redundancy number.** A small residual at low r_i is the most dangerous reading in the whole report, not the most reassuring.

The mechanism is leverage. In whitened form the redundancy number is the complement of the leverage h_i, the observation's own influence on its own fitted value:

$$r_i = 1 - h_i, \quad h_i = \mathbf{a}_i^{\mathrm{T}} \mathbf{N}^{-1} \mathbf{a}_i \tag{5.4}$$

An observation with $h_i \to 1$ determines its own fitted value: the adjustment simply follows it, so it cannot disagree with itself and its residual goes to zero whatever its error. This is geometric, not statistical -- no amount of data quality control detects it, and no test can recover information the network never collected. It is why reliability (Level 3) is reported separately from residuals (Level 2).

Lie 2 -- "the tau test found no outliers, so the data are clean"

Baarda's w divides by a KNOWN σ_0 and is normally distributed. Pope's τ divides by the ESTIMATED s_0 -- which is the honest thing to do when σ_0 is unknown, and is also the trap:

$$\tau_i = v_i \,/\, (s_0 \sqrt{r_i} \sigma_i), \quad s_0^2 = \mathbf{v}^{\mathrm{T}} \mathbf{P} \mathbf{v} \,/\, r \tag{5.5}$$

The blunders are in $\mathbf{v}$, so they are in s_0, so they are in the denominator of their own test statistic. One large error inflates s_0 enough to shrink its own τ; several inflate it enough to hide each other. Past a few per cent contamination the test goes blind, and a 100σ error can pass.

> **This is also why rejection is ONE AT A TIME.** The usual justification -- that a blunder smears its residual onto neighbours through the correlation in $\mathbf{Q}_{vv}$ -- was tested directly and is not what dominates: with a Bonferroni threshold, simultaneous rejection produced no false positives. The real reason is MASKING. Remove one blunder, s_0 drops, and the next becomes visible. Reject several at once and you are still testing against the inflated scale.

The defence is to estimate the scale robustly, so the blunders cannot set the threshold that is supposed to catch them. Read the contamination gauge first, and switch to `'test','mad'` when it is high.

5.3 The quadrant you must inspect

	low $\lvert w \rvert$ (quiet)	high $\lvert w \rvert$ (loud)
high *r* (checked)	healthy	plain outlier -- snoop it
low *r* (unchecked)	**blind spot** -- do not relax	leverage point -- a small w still hides a big error

Table 5.2 -- residual against redundancy. The two are only interpretable together.

> **The bottom row is where reputations are lost.** If a parameter you care about depends on bottom-row observations, no amount of clean residuals should reassure you. Strengthen the geometry or add independent data -- there is no statistical treatment that recovers information the network never collected.

Quantifying the blind spot: the MDB

The quadrant is qualitative; the minimal detectable bias makes it a number. Invert the test of equation 2.7: how large must an error be before it is found with probability β_0?

$$\mathrm{MDB}_i = \delta_0(\alpha, \beta_0)\, \sigma_i \,/\, \sqrt{r_i} \tag{5.6}$$

δ_0 is the non-centrality the chosen significance and power demand -- about 4.13 for $\alpha = 0.001$ and $\beta_0 = 0.80$. Everything problem-specific sits in $\sigma_i/\sqrt{r_i}$, and the square root is the whole story: at $r_i = 0.5$ the MDB is 1.4 σ_i; at 0.05 it is 4.5; at 0.01 it is 10. **Halving the redundancy number does not halve what you can detect -- it multiplies it by $\sqrt{2}$.**

> **Report MDBs, not just residuals.** An MDB says what the network CANNOT rule out, which is the question a reader of your results actually has. A table of small residuals answers a different and easier question. And compute them at the CONVERGED weights: an MDB from a mis-weighted solve is wrong by exactly the factor the variance components were about to correct.

5.4 Deciding what to do next

Level 1 failed -- blunders, or the model?

Run `lsqvce.engine.systematic(D)` *before* snooping. If the residuals show runs, serial correlation or per-group offsets, the problem is the model -- an unmodelled signal, a wrong weight ratio, coloured noise treated as white. Snooping that away merely deletes good data from around a defect that remains. Fix the model, then re-solve.

Diffuse contamination -- reject or reweight?

Isolated blunders call for iterative `lsqvce.engine.snoop`. Many small contaminations call for robust M-estimation (`lsqvce.engine.robust`), which degrades gracefully where snooping would chase its own tail. The contamination gauge tells you which regime you are in.

Group weights look wrong -- variance components

If one data type or satellite is systematically over- or under-fit, read the variance components. A factor far from 1 means that group was mis-weighted going in.

> **Re-solve at the converged weights before reading reliability.** MDBs computed at the wrong weights are wrong. Confirm the loop converged and that the minimum redundancy was met for every group -- `R.summary()` names the streams, `R.convergence()` shows whether they settled.

5.5 Reading a gravity solution

For the block-arrow path, three habits keep you honest.

1. **Diagnose through the accumulator bridge** (`lsqvce.engine.diagnoseAccum`), never the reduced common system alone. The common-parameter redundancies are only correct once the arc-local parameters have been counted; skip the bridge and every reliability number is inflated.
2. **Map the redundancy of the field**, not just its residuals. A region carried by a single arc is a blind spot: its formal error can look fine while being uncheckable.
3. **Watch for a near-singular arc.** An arc with too few observations to fix its own state poisons the Schur complement (equation 2.10). The accumulator flags it -- do not force it in.

> **A clean bill of health means all of:** Level 0 exact, Level 1 consistent, no high-w observations left, contamination low, and the parameters you care about backed by high-redundancy observations with acceptable MDB. Any one of those missing turns "passed" into "passed, but…".

6 Reference

6.1 Installation

The toolbox is plain MATLAB. There is nothing to compile, no MEX file, and no runtime dependency outside MATLAB itself -- the Python in `dev/docs_src/` builds this manual and is never called from MATLAB.

Steps

1. **Unpack** the toolbox anywhere you have write access. The folder holding `+lsqvce` is the *toolbox root*; every instruction below assumes MATLAB's working directory is that root.
2. **Run the setup** once:

```
cd /path/to/lsqvce
lsqvce_setup
```

This adds the root (making the `+lsqvce` package visible), adds `tests/`, `examples/` and `template/`, and rebuilds the Help browser index so `doc lsqvce` works. It should run **without warnings**; an `addpath` warning means a folder is missing and the unpack was incomplete.

3. **Verify**, before trusting anything:

```
run_all_lsqvce          % expect 497/498 passing, 0 failed, ~130 s
```

One test is reported *incomplete*, not failed: it is filtered by assumption because it needs a parallel pool. **0 failed** is the number that matters.

4. **Save the path** if you want it to persist across MATLAB sessions -- `lsqvce_setup` modifies the path for the current session only:

```
savepath                % optional, and it is a global change
```

Requirements

Component	Needed for	Required?
MATLAB R2019b or newer	the toolbox itself (string arrays, `arguments` blocks are not used, so older releases may work but are untested)	yes
Parallel Computing Toolbox	the parallel accumulation paths only	no -- everything falls back to serial
`reportlab` + FreeSerif fonts	rebuilding *this manual*	no -- build-time only

> **Nothing in the toolbox writes outside its own folder** except `savepath`, which you invoke deliberately, and the test log `tests/last_run_all.txt`. There is no installer, no registry entry and no network access at any point.

First steps after installing

Command	What it does
`doc lsqvce`	the landing page in the Help browser
`demo_workflow`	a guided walk-through, section by section
`demo_grace`	the worked GRACE-FO case, end to end
`edit lsqvce_template`	start your own case (Part 4)
`run_lsqvce('adjustvce')`	one test suite, while iterating

6.2 Symbols

Symbol	Meaning
n, u, r	observations, parameters, redundancy $r = n-u$

Symbol	Meaning
A, **l**, **e**	design (Jacobian), observations, errors
C, **P**	observation cofactor and weight, $\mathbf{P} = \mathbf{C}^{-1}$
σ_0^2, s_0^2	a-priori and a-posteriori variance of unit weight
N, **b**	normal matrix $\mathbf{A}^T\mathbf{P}\mathbf{A}$ and right side $\mathbf{A}^T\mathbf{P}\mathbf{l}$
$\mathbf{Q}_{xx} = \mathbf{N}^{-1}$	parameter cofactor
v, $\mathbf{Q}_{vv}$	residuals, residual cofactor
r_i, h_i	redundancy number and leverage, $r_i = 1 - h_i$ (whitened)
w_i, τ_i	Baarda standardised residual, Pope tau
n_c, n_l, K	common parameters, local parameters per arc, number of arcs
D, $\mathbf{R}_t$, d	per-epoch spatial factor, temporal correlation, number of axes

6.3 Equation summary

Eq	Subject	Section
2.1–2.3	Gauss-Markov model, BLUE, dispersion	2.1
2.4	QR of the whitened design matrix	2.2
2.5–2.7	residuals, redundancy numbers, Baarda / MDB	2.3
2.8–2.9	variance components, Helmert iteration	2.4
2.10	Schur complement, reduced common system	2.5
2.11	autoregressive noise model	2.6
2.12–2.13	separable covariance, epoch indexing	2.7
5.1	expected residual under a bias	5.2

6.4 Error identifiers you are likely to meet

Identifier	Means
`lsqvce:makeLoader:badDay`	your `buildDay` omitted a required field
`lsqvce:makeLoader:dayShape`	**A**, **B**, **l**, **t** do not share a row count
`lsqvce:covModel:badCep`	`Cep` is neither *nEpoch*×*d* nor *nEpoch*×*d*×*d*
`lsqvce:covModel:badSigma`	a separable model needs a positive scalar innovation std
`lsqvce:Results:noHistory`	you called an iteration reader on a single-solve result
`lsqvce:verbosity:badLevel`	verbosity outside 0–2 (level 3 was merged into 2)
`lsqvce_template:notImplemented`	a template stub was called before you wrote it

6.5 Testing and the suite

```
run_all_lsqvce              % the whole suite, with a full log
run_lsqvce('adjustvce')     % one suite, while iterating
run_lsqvce('*degenerate*')  % one test, wherever it lives
```

> **Do not use `runtests` on a folder.** It SKIPS any file that fails to load and still reports the rest as green, so the suite can look clean while an entire file never ran. `run_all_lsqvce` iterates the registered list, counts a load failure as a failure, keeps the warning inventory (warnings are a signal, not noise) and records per-suite seconds.

The suite deliberately provokes warnings in its failure-mode tests, so a warning-free run would itself be suspicious. Compare the warning inventory and the per-suite timings between runs, not just the pass count.

References

[1] Baarda, W. (1968). *A Testing Procedure for Use in Geodetic Networks.* Netherlands Geodetic Commission, Publications on Geodesy, New Series, 2(5). Delft.

[2] Pope, A. J. (1976). *The Statistics of Residuals and the Detection of Outliers.* NOAA Technical Report NOS 65 NGS 1. Rockville, MD.

[3] Koch, K.-R. (1999). *Parameter Estimation and Hypothesis Testing in Linear Models*, 2nd ed. Springer, Berlin.

[4] Teunissen, P. J. G. (2006). *Testing Theory: an Introduction*, 2nd ed. VSSD, Delft.

[5] Teunissen, P. J. G. (2003). *Adjustment Theory: an Introduction.* VSSD, Delft.

[6] Förstner, W. (1979). Ein Verfahren zur Schätzung von Varianz- und Kovarianzkomponenten. *Allgemeine Vermessungs-Nachrichten*, 86, 446–453.

[7] Helmert, F. R. (1907). *Die Ausgleichungsrechnung nach der Methode der kleinsten Quadrate*, 2nd ed. Teubner, Leipzig.

[8] Grafarend, E. W. & Schaffrin, B. (1993). *Ausgleichungsrechnung in linearen Modellen.* BI-Wissenschaftsverlag, Mannheim.

[9] Koch, K.-R. & Kusche, J. (2002). Regularization of geopotential determination from satellite data by variance components. *Journal of Geodesy*, 76(5), 259–268.

[10] Kusche, J. (2003). A Monte-Carlo technique for weight estimation in satellite geodesy. *Journal of Geodesy*, 76(11–12), 641–652.

[11] Golub, G. H. & Van Loan, C. F. (2013). *Matrix Computations*, 4th ed. Johns Hopkins University Press, Baltimore.

[12] Hutchinson, M. F. (1990). A stochastic estimator of the trace of the influence matrix for Laplacian smoothing splines. *Communications in Statistics -- Simulation and Computation*, 19(2), 433–450.

[13] Golub, G. H., Heath, M. & Wahba, G. (1979). Generalized cross-validation as a method for choosing a good ridge parameter. *Technometrics*, 21(2), 215–223.

[14] Huber, P. J. (1981). *Robust Statistics.* Wiley, New York.

[15] Levinson, N. (1947). The Wiener RMS error criterion in filter design and prediction. *Journal of Mathematics and Physics*, 25, 261–278.

[16] Durbin, J. (1959). Efficient estimation of parameters in moving-average models. *Biometrika*, 46(3/4), 306–316.

[17] Ljung, G. M. & Box, G. E. P. (1978). On a measure of lack of fit in time series models. *Biometrika*, 65(2), 297–303.

[18] Wen, H. Y., Kruizinga, G., Paik, M., et al. (2019). *GRACE-FO Level-1 Data Product User Handbook.* JPL D-56935, NASA Jet Propulsion Laboratory.

[19] Kornfeld, R. P., Arnold, B. W., Gross, M. A., et al. (2019). GRACE-FO: the Gravity Recovery and Climate Experiment Follow-On mission. *Journal of Spacecraft and Rockets*, 56(3), 931–951.

[20] Schaffrin, B. (1997). Reliability measures for correlated observations. *Journal of Surveying Engineering*, 123(3), 126–137.

[21] Ellmer, M. (2018). *Contributions to GRACE Gravity Field Recovery: Improvements in Dynamic Orbit Integration, Stochastic Modelling of Observations, and Variance Component Estimation.* PhD thesis, TU Graz.

[22] Kvas, A. (2019). *Stochastic Modelling in Satellite Gravimetry.* PhD thesis, TU Graz.

---

lsqvce 1.0 -- consolidated diagnostics and streaming variance component estimation. This manual was generated with assistance from Claude (Anthropic), model Claude Opus 4.8. Build it with `python3 dev/docs_src/build_master_pdf.py`; see `dev/docs_src/README_manual.md`.